

GUIA DO PROJETO PRÁTICO:

Ecosistema de E-commerce Distribuído

Um roteiro passo a passo para construir integrações modernas

Data: 15 de Maio de 2024 | Versão 1.0

❖❖ Visão Geral do Projeto: Construindo um E-commerce Integrado

O objetivo principal **não é criar um e-commerce completo** com todas as funcionalidades de negócio (carrinho de compras, login de usuário, etc.). Nosso foco será **exclusivamente nas estratégias de integração**: como os serviços conversam entre si, como lidam com falhas e como garantem que os dados fluam de forma eficiente e segura.

Este é o seu momento de brilhar e aplicar os padrões de comunicação síncrona (APIs REST/gRPC) e assíncrona (Mensageria), além de implementar mecanismos de resiliência que são cruciais em qualquer sistema distribuído.

❖❖ Objetivos de Aprendizagem

Ao final deste projeto, você será capaz de:

- Projetar e implementar APIs para comunicação síncrona entre serviços.
- Utilizar mensageria para comunicação assíncrona e desacoplamento de sistemas.
- Implementar padrões de resiliência como Circuit Breaker e Dead Letter Queue.
- Compreender e aplicar os conceitos de idempotência e tratamento de falhas.
- Trabalhar com ferramentas como Docker para orquestração de ambientes.

❖❖ Ferramentas e Tecnologias (Sugestões)

- Linguagem de Programação: Escolha a que você e seu grupo se sentirem mais confortáveis (Python, Node.js, Java, C#, Go, etc.). O foco é a integração, não a linguagem em si.
- Contratos de API: OpenAPI/Swagger para REST, Protocol Buffers para gRPC.
- Message Broker: RabbitMQ (via Docker é a forma mais fácil), ou simulação com AWS SQS/SNS se tiverem familiaridade.
- Containerização: Docker e Docker Compose são altamente recomendados para configurar o ambiente de forma rápida e consistente.

❖❖ Arquitetura do Ecosistema de E-commerce

Distribuído

Nosso e-commerce será composto por **quatro microsserviços principais**, cada um com uma responsabilidade bem definida. A beleza está em como eles se conectam!

1.

Checkout API

- **Função:** É a porta de entrada do nosso sistema. Recebe as requisições de compra dos clientes.
- **Integrações:** Comunica-se sincronamente com o Serviço de Estoque para verificar a disponibilidade dos produtos.
- **Comunica-se assincronamente** com os demais serviços (Pagamento e Notificação) publicando eventos de "Pedido Criado".

2.

Serviço de Estoque

- **Função:** Gerencia a quantidade de produtos disponíveis.
- **Integrações:** Recebe requisições síncronas do Checkout API para verificar e reservar itens.
- **Dados:** Pode ser um simples dicionário em memória ou um arquivo JSON para simular o estoque. Não precisa de banco de dados complexo.

3.

Serviço de Pagamento

- **Função:** Simula o processamento de um pagamento.
- **Integrações:** Consome eventos assíncronos de "Pedido Criado" publicados pelo Checkout API.
- **Dados:** Pode simular sucesso ou falha aleatoriamente.

4.

Serviço de Notificação

- **Função:** Simula o envio de uma confirmação de pedido (ex: e-mail, SMS).
- **Integrações:** Consome eventos assíncronos de "Pedido Criado" publicados pelo Checkout API.
- **Resiliência:** Deve ser configurado para usar uma Dead Letter Queue (DLQ) caso falhe repetidamente.

❖❖ Roteiro de Implementação: Marcos de Entrega

O projeto será dividido em três marcos principais, que coincidem com as avaliações da disciplina. Cada marco adiciona uma camada de complexidade e novos padrões de integração.

❖❖ **Marco 1: Comunicação Síncrona (Checkout & Estoque)** Este é o ponto de partida! Vamos fazer os dois primeiros serviços conversarem diretamente.

- **Objetivo:** Implementar a

comunicação síncrona entre o Checkout API e o Serviço de Estoque.

- Passos de Implementação: Configuração do Ambiente: Crie um repositório Git para o seu projeto.
- Configure o ambiente de desenvolvimento para cada serviço (ex: package.json para Node.js, requirements.txt para Python).
- Dica: Utilize Docker Compose para orquestrar os dois serviços. Isso facilitará muito a vida!
- Serviço de Estoque: Crie uma API simples (REST ou gRPC) que exponha um endpoint para verificar a disponibilidade de um produto.
- Exemplo de endpoint REST: GET /produtos/{id}/disponibilidade ou POST /estoque/verificar
- A lógica de estoque pode ser um simples mapa em memória: {"produto_A": 10, "produto_B": 5}.
- Retorne um status HTTP adequado (ex: 200 OK com disponivel: true ou 404 Not Found se o produto não existir, 400 Bad Request se a quantidade for inválida).
- Checkout API: Crie uma API que receba requisições de compra (ex: POST /pedidos).
- Antes de "finalizar" o pedido, o Checkout deve fazer uma requisição síncrona para o Serviço de Estoque para verificar a disponibilidade dos produtos.
- Se o estoque estiver disponível, o Checkout pode simular a criação do pedido (apenas logar no console). Se não, retorne um erro adequado ao cliente.
- Contratos de API: Documente os contratos das APIs do Checkout e do Estoque usando OpenAPI/Swagger (para REST) ou Protocol Buffers (para gRPC).
- Entregáveis para N1: Código-fonte dos serviços Checkout API e Serviço de Estoque.
- Documentação dos contratos de API.
- Demonstração da comunicação síncrona funcionando (Checkout chamando Estoque). ⚡

Marco 2: Comunicação Assíncrona e Resiliência

Agora vamos adicionar complexidade e robustez ao nosso sistema, introduzindo mensageria e padrões de resiliência.

- Objetivo: Implementar comunicação assíncrona via Message Broker e adicionar padrões de resiliência (Circuit Breaker e Dead Letter Queue).
- Passos de Implementação: Configuração do Message Broker: Adicione um Message Broker (ex: RabbitMQ) ao seu docker-compose.yml.
- Configure os serviços para se conectarem a ele.
- Checkout API (Atualização): Após verificar o estoque e "criar" o pedido, o Checkout não chamará diretamente os serviços de Pagamento e Notificação. Em vez disso, ele publicará um evento "PedidoCriado" no Message Broker.
- O evento deve conter informações essenciais do pedido (ID do pedido, itens, valor, etc.).
- Implemente um Circuit Breaker na chamada síncrona do Checkout para o Serviço de Estoque. Se o Estoque falhar repetidamente, o Circuit Breaker deve "abrir" e o Checkout deve falhar rapidamente sem tentar chamar o Estoque.

- Serviço de Pagamento: Crie este novo serviço.
 - Ele deve consumir eventos "PedidoCriado" do Message Broker.
 - Ao receber um evento, simule o processamento do pagamento (pode ser um setTimeout com sucesso/falha aleatória). Logue o resultado.
 - Serviço de Notificação: Crie este novo serviço.
 - Ele também deve consumir eventos "PedidoCriado" do Message Broker.
 - Ao receber um evento, simule o envio de uma notificação (ex: logar "E-mail enviado para o cliente X").
 - Implemente uma Dead Letter Queue (DLQ): Configure o serviço de Notificação para que, se ele falhar ao processar uma mensagem (ex: erro interno, mensagem malformada) após um número configurável de tentativas, a mensagem seja movida para uma DLQ.
 - Entregáveis para N2: Código-fonte de todos os 4 serviços.
 - docker-compose.yml configurado com todos os serviços e o Message Broker.
- Demonstração: Checkout publicando eventos.
- Pagamento e Notificação consumindo eventos.
 - Circuit Breaker funcionando (simule a queda do Serviço de Estoque).
 - Mensagens indo para a DLQ (simule falhas repetidas no Serviço de Notificação). ❓❓

Marco 3: Entrega Final e Apresentação

Este é o momento de consolidar tudo, refinar e apresentar seu trabalho!

- Objetivo: Apresentar um ecossistema de e-commerce integrado, robusto e resiliente, demonstrando a compreensão dos padrões de integração.
 - Passos de Implementação: Revisão e Refinamento: Revise todo o código. Garanta que a manipulação de erros seja consistente em todos os serviços.
 - Verifique se todos os requisitos dos Marcos 1 e 2 estão funcionando perfeitamente.
 - Adicione logs claros em todos os serviços para facilitar a depuração e a compreensão do fluxo.
 - Documentação: Crie um README.md detalhado no seu repositório Git, explicando como configurar e rodar o projeto.
 - Inclua um diagrama de arquitetura simples (pode ser um desenho à mão digitalizado ou feito em ferramentas como draw.io).
 - Descreva as decisões de design e os padrões de integração utilizados.
 - Preparação para a Apresentação: Prepare uma breve apresentação (5-10 minutos) para a turma, explicando a arquitetura, as tecnologias usadas e demonstrando o funcionamento do sistema.
 - Esteja pronto para responder perguntas sobre as escolhas de design e como os padrões de resiliência foram implementados.
 - Entregáveis para N3: Repositório Git com todo o código-fonte e README.md completo.
- Apresentação oral e demonstração do projeto funcionando.

❓❓ Dicas Essenciais para o Sucesso

- Comece Cedo: Integração é um desafio. Não deixe para a última hora!
- Foco na Integração: Lembre-se, o objetivo é a integração, não a complexidade da regra de negócio. Mockar dados e simular comportamentos é perfeitamente aceitável e até encorajado para manter o foco.
- Docker é Seu Amigo: Use Docker e Docker Compose desde o início. Isso padroniza o ambiente e evita problemas de "na minha máquina funciona".
- Testes Simples: Teste cada integração isoladamente antes de juntar tudo. Use ferramentas como Postman ou Insomnia para testar suas APIs.
- Comunicação no Grupo: Trabalhem juntos, dividam as tarefas e ajudem-se mutuamente. A engenharia de software é um esporte de equipe!
- Pergunte! Não hesite em perguntar ao professor ou aos colegas se tiver dúvidas. Este é um ambiente de aprendizado.

Este projeto é a sua chance de construir algo real e tangível, aplicando conhecimentos que são altamente valorizados no mercado de trabalho. Boa sorte e divirtam-se codificando!

"A integração não é sobre conectar sistemas, é sobre conectar pessoas e processos."